

De Verborgen QR-code

Een puzzel in lineaire algebra over $\text{GF}(2)$

*Achter twee matrices van 29 bij 29 nullen en enen
schuilt een QR-code die naar iets leidt.
De enige weg erheen: Gauss-eliminatie, volledig modulo 2.*

Inhoud

1	De opdracht — overzicht	2
2	Fase 1 — Het sleutelgetal	3
2.1	De taylorreeks van de arctangens	3
2.2	Convergentie op de rand $x = 1$ (stelling van Abel)	3
2.3	Het sleutelgetal berekenen	3
2.4	Omzetten naar binaire vorm	3
3	Fase 2 — Van sleutelgetal naar matrices: het LFSR	4
3.1	Wat is een LFSR?	4
3.2	Volledig uitgewerkt minivoorbeeld (register van lengte 4)	4
3.3	Van bitreeks naar matrices	4
4	Fase 3 — De vercijferde matrix demaskeren	6
5	Fase 4 — Gauss-Jordan-eliminatie modulo 2	7
5.1	Het principe	7
5.2	Het algoritme, stap voor stap	7
5.3	Volledig uitgewerkt minivoorbeeld (3×3)	7
6	Van oplossing naar scanbare code	9
7	Oplossingssleutel	10

1 De opdracht — overzicht

Deze keer krijg je matrix A niet cadeau. Je moet ze zelf opbouwen. Het traject verloopt in vier fasen:

1. **Los een vergelijking op** om een geheim *sleutelgetal* S te vinden (een getal tussen 0 en $2^{29} - 1$).
2. **Voer S in een schuifregister** (een LFSR — meer hierover in §3) en genereer daarmee, bit voor bit, twee matrices: matrix A zelf, én een *maskeermatrix* M .
3. Je krijgt een **vercijferde matrix** B_{enc} . Met je zelfgemaakte M demaskeer je die tot de echte matrix B .
4. Los tot slot, zoals eerder, het stelsel $A \cdot X \equiv B \pmod{2}$ op via Gauss-eliminatie. De oplossing X is opnieuw een 29×29 -matrix van 0'en en 1'en die — zwart voor 1, wit voor 0, met een witte rand — een echte, scanbare QR-code oplevert.

2 Fase 1 — Het sleutelgetal

Bereken:

$$S = \frac{2000}{\pi} \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}.$$

2.1 De Taylorreeks van de arctangens

We vertrekken van de bekende meetkundige reeks

$$\frac{1}{1+t^2} = \sum_{n=0}^{\infty} (-1)^n t^{2n}, \quad |t| < 1.$$

Dit is simpelweg de meetkundige reeks $\frac{1}{1-r} = \sum r^n$ met $r = -t^2$. Integreren van 0 tot x aan beide zijden (toegelaten binnen het convergentie-interval, term voor term):

$$\arctan(x) = \int_0^x \frac{dt}{1+t^2} = \sum_{n=0}^{\infty} (-1)^n \int_0^x t^{2n} dt = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{2n+1}, \quad |x| < 1.$$

2.2 Convergentie op de rand $x = 1$ (stelling van Abel)

De afgeleide reeks $\sum \frac{(-1)^n x^{2n+1}}{2n+1}$ convergeert voor $|x| < 1$, maar we willen ze evalueren in $x = 1$, precies op de rand van het convergentiegebied. Bij $x = 1$ wordt de reeks $\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$, een alternerende reeks met termen die monotoon naar 0 dalen — deze convergeert (via het alternerend-reeksencriterium van Leibniz). De stelling van Abel garandeert dat wanneer een machtreeks op de rand van haar convergentie-interval convergeert, haar som daar gelijk is aan de limiet van de functie vanuit het binnenste:

$$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = \lim_{x \rightarrow 1^-} \arctan(x) = \arctan(1) = \frac{\pi}{4}.$$

2.3 Het sleutelgetal berekenen

$$S = \frac{2000}{\pi} \cdot \frac{\pi}{4} = \frac{2000}{4} = 500.$$

Controle: S moet een geheel getal zijn tussen 0 en $2^{29} - 1 = 536\,870\,911$ — dat klopt hier ruimschoots.

2.4 Omzetten naar binaire vorm

Voor de volgende fase hebben we $S = 500$ nodig als een rij van precies 29 bits (aangevuld met voorloopnullen):

$$500_{10} = 00000000000000000000111110100_2.$$

(Controle: $111110100_2 = 256 + 128 + 64 + 32 + 16 + 4 = 500$.)

3 Fase 2 — Van sleutelgetal naar matrices: het LFSR

3.1 Wat is een LFSR?

Een **lineair-terugkoppelingsschuifregister** (Engels: *linear feedback shift register*, LFSR) is een eenvoudig mechanisme dat een oneindige reeks bits genereert uit een eindig geheugen. Het bestaat uit een rij van d geheugenposities (hier $d = 29$), gevuld met een startwaarde (de *seed*). Bij elke stap:

1. Bereken een **terugkoppelbit**: de XOR van de bits op vaste, vooraf afgesproken posities (de *tap-posities*).
2. Schuif alle bits één positie op.
3. De nieuwe terugkoppelbit komt binnen aan het ene uiteinde; de bit die aan het andere uiteinde “uitvalt” is de **output-bit** van deze stap.

Dit is exact dezelfde soort wiskunde (lineaire recursies modulo 2) als bij foutcorrectie in QR-codes en bij stroomvercijfering in cryptografie — het past dus naadloos bij de rest van deze puzzel.

Voor deze puzzel: register van lengte $d = 29$, tap-posities 29 en 2 (dit hoort bij de *primitieve veelterm* $x^{29} + x^2 + 1$ over $\text{GF}(2)$ — een standaardkeuze uit de codeertheorie die garandeert dat de reeks pas na $2^{29} - 1$ stappen zichzelf herhaalt).

3.2 Volledig uitgewerkt minivoorbeeld (register van lengte 4)

Ter illustratie: een LFSR met $d = 4$, tap-posities 4 en 1, en seed $1001_2 = 9$. Het register wordt weergegeven als (r_1, r_2, r_3, r_4) , met r_1 het meest linkse bit.

stap	register vóór de stap	output-bit (= r_4 vóór schuiven)
1	(1, 0, 0, 1)	1
2	(0, 1, 0, 0)	0
3	(0, 0, 1, 0)	0
4	(0, 0, 0, 1)	1
5	(1, 0, 0, 0)	0
6	(1, 1, 0, 0)	0
7	(1, 1, 1, 0)	0
8	(1, 1, 1, 1)	1

Toelichting bij stap 1: terugkoppelbit = $r_4 \oplus r_1 = 1 \oplus 1 = 0$. De output-bit van deze stap is het bit dat rechts uitvalt: $r_4 = 1$. Na het schuiven wordt het nieuwe register (0, 1, 0, 0) — de terugkoppelbit (0) komt links binnen, en r_1, r_2, r_3 schuiven één plaats op.

Toelichting bij stap 4: register is (0, 0, 0, 1). Terugkoppelbit = $r_4 \oplus r_1 = 1 \oplus 0 = 1$. Output-bit = $r_4 = 1$. Nieuw register: (1, 0, 0, 0).

De resulterende bitreeks na 8 stappen: 1, 0, 0, 1, 0, 0, 0, 1, ... Dit proces zet je nu voort voor de echte puzzel — maar met een register van lengte 29, tap-posities 29 en 2, en de seed uit Fase 1.

3.3 Van bitreeks naar matrices

Start het register met de 29-bits seed $500_{10} = 00000000000000000000111110100_2$ (het meest linkse bit is r_1). Genereer op de hierboven beschreven manier een reeks van minstens 1682 output-bits.

- De **eerste 841 bits** vul je, rij voor rij, in een rooster van 29×29 — dit wordt matrix A .
- De **volgende 841 bits** (bit 842 t.e.m. 1682) vul je op dezelfde manier in een tweede rooster van 29×29 — dit wordt de maskeermatrix M .

Zelfcontrole. Tel, ter controle, het totaal aantal enen in je matrices:

- Matrix A moet exact **421** enen bevatten (op 841 cellen).
- Matrix M moet exact **416** enen bevatten (op 841 cellen).

Klopt dit niet? Zoek dan eerst de fout in je bitreeks voor je verdergaat — een enkele omgewisselde bit vroeg in de reeks verschuift alles wat erna komt.

4 Fase 3 — De vercijferde matrix demaskeren

Je krijgt hieronder niet matrix B zelf, maar een **vercijferde versie** B_{enc} . Deze is ontstaan door de echte matrix B bit voor bit te XOR-en met jouw maskeermatrix M :

$$B_{\text{enc}} = B \oplus M \pmod{2}.$$

Omdat XOR zijn eigen inverse is ($a \oplus b \oplus b = a$), geldt: zodra je M hebt (uit Fase 2), demaskeer je met exact dezelfde operatie:

$$B = B_{\text{enc}} \oplus M \pmod{2}.$$

Bereken dit cel voor cel (841 keer XOR) om de echte matrix B te bekomen.

De echte matrix B moet, ter controle, exact **459** enen bevatten.

De vercijferde matrix B_{enc}

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
1	0	0	1	1	0	0	1	0	1	0	0	0	0	1	1	0	1	1	1	1	0	1	0	1	1	0	0	0	0
2	1	0	0	1	1	1	0	1	1	1	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1	0	0	1	0
3	0	0	1	0	1	1	0	0	0	0	1	0	1	0	1	1	1	1	0	1	0	0	1	0	0	1	0	1	1
4	0	0	0	1	1	0	1	0	1	0	0	1	0	0	1	1	0	1	1	1	0	0	0	0	0	0	1	1	0
5	1	0	1	0	0	0	1	0	1	0	1	0	0	0	0	1	1	1	0	1	0	0	0	1	1	1	1	0	0
6	1	1	0	0	1	1	0	0	0	0	1	1	0	1	1	0	1	1	0	1	0	1	1	1	1	0	0	0	1
7	0	1	1	0	1	1	0	0	0	0	1	0	0	1	0	1	0	1	1	0	0	1	0	0	1	1	1	0	1
8	0	0	0	1	0	1	1	1	1	0	1	0	0	0	0	0	1	1	1	0	0	1	0	1	1	0	1	1	0
9	1	1	1	1	0	1	0	1	1	1	0	0	1	0	1	0	0	0	1	0	1	0	0	1	0	0	0	0	1
10	0	1	0	1	1	1	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	1	0	1	1	1	0	0	1
11	1	0	1	0	0	1	0	0	0	0	1	1	0	1	1	1	1	0	0	1	1	1	0	0	0	0	1	0	1
12	1	0	0	0	0	1	1	0	0	1	0	0	0	1	1	1	0	1	0	1	1	1	0	0	1	1	0	1	0
13	1	0	1	0	1	1	1	1	0	0	0	1	1	0	1	0	0	1	0	1	0	0	1	1	1	1	0	0	0
14	0	1	1	1	1	0	0	1	0	1	0	1	0	0	0	0	1	0	1	1	0	1	1	1	0	1	0	1	1
15	0	0	0	0	0	0	1	1	1	1	1	0	1	1	0	1	1	1	0	0	0	1	0	0	0	0	1	0	1
16	1	1	1	1	0	1	1	0	1	1	1	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0
17	0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	1	1	1	1	0	1	1	1	1	0	0	1	0	0
18	1	0	0	0	0	1	0	1	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1	1	0	1	0	0	1
19	1	1	0	0	1	0	0	1	0	1	0	0	0	1	1	0	1	0	0	0	1	0	1	1	0	0	1	1	1
20	0	0	1	1	1	0	1	1	0	1	1	0	0	1	0	1	0	0	1	0	1	0	0	1	0	0	1	0	0
21	1	0	1	0	0	0	1	1	0	1	1	1	0	1	1	0	0	1	1	0	1	0	1	1	1	0	1	1	1
22	1	1	1	0	1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0
23	1	0	0	0	0	1	1	1	1	1	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	1	1
24	0	0	1	0	0	1	0	1	0	0	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	1	1	1	1
25	1	0	0	1	1	1	1	0	1	1	0	1	1	0	0	1	0	1	1	0	0	0	1	0	0	1	0	1	0
26	0	0	0	0	0	1	0	1	1	1	0	0	0	1	1	1	0	1	1	0	1	1	1	0	1	0	1	0	1
27	0	1	0	1	1	0	0	0	0	0	1	0	1	0	0	0	1	0	0	0	1	0	0	0	1	1	1	0	0
28	0	0	0	0	0	1	0	1	0	0	1	0	0	0	1	1	1	0	0	0	0	0	1	0	0	0	0	1	1
29	1	0	1	1	0	1	1	1	1	1	1	0	0	1	1	0	0	0	1	0	1	1	0	0	1	0	1	1	0

5 Fase 4 — Gauss-Jordan-eliminatie modulo 2

Nu je zelf matrix A hebt opgebouwd (Fase 2) en matrix B hebt gedemaskeerd (Fase 3), is de rest identiek aan een klassiek stelsel: vind de unieke matrix X (opnieuw 29×29 , enkel 0'en en 1'en) die voldoet aan

$$A \cdot X \equiv B \pmod{2}.$$

Wanneer je X correct hebt gevonden en de 1'en zwart kleurt en de 0'en wit laat (met een witte rand eromheen), bekom je een **echte, scanbare QR-code**.

Waarom dit werkt. Rekenen “modulo 2” betekent: elke optelling levert een 0 of een 1 op, nooit meer. Concreet is $1 + 1 = 0$, $1 + 0 = 1$, $0 + 0 = 0$ — dit is precies de logische XOR-operatie. Matrices met enkel 0'en en 1'en, waarbij alle rekenwerk op deze manier gebeurt, leven in wat wiskundigen het **eindig veld** $\text{GF}(2)$ noemen (Galois-veld met 2 elementen). Alle vertrouwde regels van lineaire algebra — matrixvermenigvuldiging, Gauss-eliminatie, inverteerbaarheid — blijven gewoon geldig, alleen de optelling verandert. Deze wiskunde is de basis van foutcorrectie in QR-codes zelf, van cryptografie, en van coderingstheorie in het algemeen.

5.1 Het principe

Om X te vinden uit $AX \equiv B \pmod{2}$, bouw je de **uitgebreide matrix** $[A \mid B]$ (matrix A en matrix B naast elkaar geplakt, samen dus 29×58). Door rijoperaties toe te passen — steeds modulo 2 — herleid je het linkerdeel A tot de identiteitsmatrix I . Wat er dan rechts overblijft, is exact de gezochte matrix X :

$$[A \mid B] \xrightarrow{\text{rijoperaties mod } 2} [I \mid X].$$

Toegelaten rijoperaties:

- (i) Twee rijen verwisselen.
- (ii) Eén rij optellen (XOR) bij een andere rij.

Vermenigvuldigen van een rij met een constante is hier overbodig: de enige niet-triviale constante modulo 2 is 1 zelf, dus deze operatie doet niets.

5.2 Het algoritme, stap voor stap

1. Start bij kolom 1, rij 1.
2. Zoek in de huidige kolom (op of onder de huidige rij) een rij met een 1 in die kolom — dit wordt de **pivotrij**. Is er geen enkele 1 te vinden, ga naar de volgende kolom.
3. Verwissel indien nodig deze pivotrij met de huidige rij, zodat de pivot op de diagonaal staat.
4. Tel deze pivotrij (XOR) op bij *elke andere rij* die in deze kolom een 1 heeft — zowel erboven als eronder. Na deze stap staat er in de hele kolom nergens anders meer een 1 dan in de pivotrij zelf.
5. Ga naar de volgende rij en kolom, herhaal.
6. Na 29 succesvolle pivotstappen staat links I , en rechts staat X .

5.3 Volledig uitgewerkt minivoorbeeld (3×3)

Om de methode te illustreren lossen we eerst een kleine versie op. Stel:

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}.$$

Uitgebreide matrix:

$$[A \mid B] = \left(\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \end{array} \right)$$

Stap 1 — kolom 1. Rij 1 heeft al een 1 op de diagonaal (pivot gevonden, geen verwisseling nodig). Tel rij 1 op bij elke andere rij die in kolom 1 een 1 heeft: dat is rij 3.

$$R_3 \leftarrow R_3 + R_1 = (1+1, 0+1, 1+0 \mid 1+1, 1+0, 1+1) = (0, 1, 1 \mid 0, 1, 0)$$

Nieuwe matrix:

$$\left(\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{array} \right)$$

Stap 2 — kolom 2. Rij 2 heeft een 1 op de diagonaal. Tel rij 2 op bij elke andere rij met een 1 in kolom 2: dat zijn rij 1 én rij 3.

$$R_1 \leftarrow R_1 + R_2 = (1+0, 1+1, 0+1 \mid 1+0, 0+1, 1+0) = (1, 0, 1 \mid 1, 1, 1)$$

$$R_3 \leftarrow R_3 + R_2 = (0+0, 1+1, 1+1 \mid 0+0, 1+1, 0+0) = (0, 0, 0 \mid 0, 0, 0)$$

Nieuwe matrix:

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right)$$

Hier zien we dat rij 3 volledig nul is geworden in het linkerdeel — dit *specifieke* kleine voorbeeld heeft dus een niet-inverteerbare A (rang 2 in plaats van 3), en levert dus geen unieke oplossing. Dit minivoorbeeld illustreert enkel de rijoperaties zelf; in de echte puzzel hieronder is gegarandeerd dat A volledig inverteerbaar is (rang 29), dus daar loop je dit probleem niet tegen het lijf — elke kolom krijgt gegarandeerd een pivot.

Controleer je werk onderweg. Na elke pivotstap mag je gerust teruggaan en met de hand $A \cdot X \stackrel{?}{=} B$ verifiëren op een paar losse rijen — dat is veel sneller dan de hele eliminatie opnieuw te doen, en vangt rekenfouten vroeg op.

6 Van oplossing naar scanbare code

Zodra je de volledige matrix X hebt (29 rijen, 29 kolommen, enkel 0'en en 1'en), zet je ze om in een echte QR-code:

1. Teken een raster van 29×29 vierkante cellen.
2. Kleur elke cel waar $X_{ij} = 1$ staat **zwart**; laat elke cel waar $X_{ij} = 0$ staat **wit**.
3. Voeg rondom het volledige raster een **witte rand** toe van minstens 4 cellen breed (de zogenaamde “quiet zone” — zonder deze rand herkennen de meeste camera's de code niet).
4. Print of teken dit zo dat elke cel minstens ongeveer 2–3 mm groot is — kleiner wordt onbetrouwbaar om te scannen.
5. Scan met een gewone smartphonecamera.

Ingebouwde foutmarge. Deze QR-code is aangemaakt met foutcorrectieniveau **Q** ($\sim 25\%$ redundantie). Dat betekent: zelfs als er tijdens het met de hand overtekenen van de matrix ergens een klein aantal cellen verkeerd staan, zal de code hoogstwaarschijnlijk toch nog correct scannen. Perfectie is dus niet vereist — wel nauwkeurigheid.

7 Oplossingssleutel

Enkel ter controle — probeer eerst zelf te lossen voor je verder leest.

Fase 1 — sleutelgetal

$$S = 500 = 00000000000000000000111110100_2.$$

Fase 2 — matrix A en maskeermatrix M

Matrix A (421 enen)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
1	0	0	1	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
2	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
3	0	0	1	0	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0		
4	1	0	0	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1		
5	1	1	1	1	0	0	1	0	0	1	1	1	0	0	1	0	0	1	1	1	0	0	1	0	0	1	1	1	0		
6	0	1	1	0	1	0	0	0	0	1	1	0	1	0	0	0	0	1	1	0	1	0	0	0	0	0	1	1	0	1	
7	0	0	1	0	0	0	0	0	0	1	1	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0	
8	1	0	0	0	0	0	0	0	0	1	1	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0	1	1	0	1	
9	1	1	1	1	1	1	1	1	1	0	0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	
10	1	0	0	1	1	0	0	1	1	1	1	1	0	0	0	1	1	0	0	1	1	0	0	1	1	1	1	1	1	0	
11	0	0	0	1	1	1	1	0	0	1	1	0	1	0	1	1	0	1	0	0	1	0	1	1	0	0	1	1	1	1	
12	1	1	1	0	0	1	1	1	1	0	0	0	1	0	0	1	0	0	0	0	0	1	0	0	1	0	1	1	0	0	
13	1	1	0	1	0	0	1	1	0	1	0	1	1	1	1	0	1	0	1	0	0	0	0	0	1	0	0	1	0	1	
14	1	0	1	1	1	1	0	0	0	1	0	0	1	1	0	1	1	1	0	1	0	1	0	0	0	0	0	1	0	0	
15	1	0	0	1	1	0	1	0	1	1	1	1	0	0	0	1	1	0	1	1	1	0	1	0	1	0	0	0	0	0	
16	1	0	1	1	0	1	1	1	0	0	1	1	1	1	1	0	0	0	0	1	1	0	1	1	1	0	1	0	1	0	
17	0	0	1	1	1	0	0	1	0	1	1	0	0	1	1	1	1	1	0	0	0	0	1	1	0	1	1	1	0	1	
18	0	1	1	0	0	0	0	1	0	0	1	0	1	1	0	0	1	1	1	1	1	0	0	0	0	1	1	0	1	1	
19	1	0	0	0	0	0	0	1	0	1	1	1	0	0	0	0	1	1	0	0	1	0	1	0	0	1	0	0	0	1	
20	1	1	1	1	1	1	1	0	1	1	0	0	0	0	0	0	0	1	1	1	1	0	1	1	1	1	0	1	0	0	
21	1	1	0	0	1	1	0	1	1	0	1	0	1	0	1	0	0	1	1	0	1	1	0	0	1	0	0	0	0	0	
22	1	1	1	1	0	0	0	1	1	1	0	1	1	1	0	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	
23	0	0	1	1	1	1	1	0	0	1	0	0	1	1	1	0	1	0	0	0	0	0	1	0	0	1	1	0	0	1	
24	0	1	1	0	0	1	1	1	1	0	1	0	0	1	1	1	0	1	0	1	0	1	0	0	0	0	1	1	1	1	0
25	1	1	0	1	0	0	1	1	0	1	1	1	1	0	0	1	0	0	0	1	0	1	0	1	0	1	1	0	0	1	0
26	0	1	0	0	0	0	1	1	1	0	0	1	1	1	1	0	1	0	1	1	1	0	1	1	0	1	0	0	0	0	
27	0	1	0	1	0	1	1	0	0	0	0	1	1	0	0	0	1	0	0	1	1	1	0	0	0	1	0	1	0	0	
28	1	1	1	0	1	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	1	1	0	1	0	1	1	1	0	1	
29	1	0	0	0	1	1	1	0	1	1	1	1	0	1	1	1	1	1	1	0	0	0	0	1	0	0	1	1	1	0	0

Maskeermatrix M (416 enen)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
1	0	0	0	0	1	1	0	1	1	0	0	1	0	0	1	1	0	0	1	0	1	0	0	0	0	1	1	0	1
2	0	1	0	1	1	0	1	1	0	1	0	0	0	0	1	1	1	1	0	1	1	1	1	1	1	0	0	0	1
3	0	0	0	1	1	1	0	0	0	1	0	1	0	1	1	0	0	1	0	0	1	1	0	0	1	0	1	0	0
4	0	0	0	1	1	0	1	0	1	1	1	0	1	1	0	1	0	0	0	0	1	1	1	1	1	0	1	1	1
5	1	1	1	0	0	0	1	0	0	1	1	1	0	0	0	1	0	1	0	1	1	0	0	1	0	0	1	1	0
6	0	1	1	1	1	1	0	1	0	0	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	1	0	1
7	0	0	1	1	0	0	0	1	0	1	1	0	0	1	1	1	0	0	1	0	1	0	1	1	1	1	0	0	0
8	0	1	1	0	1	0	1	1	1	0	0	0	0	1	1	0	1	0	0	0	1	0	0	1	1	1	1	1	0
9	1	1	0	1	1	1	0	0	1	0	1	0	1	1	0	1	1	1	1	1	0	1	0	0	1	1	0	0	0
10	1	1	1	0	0	1	0	1	1	1	0	1	1	0	1	1	0	0	1	1	1	0	1	0	0	1	0	1	0
11	0	1	1	1	1	0	1	1	0	0	0	1	1	1	0	0	0	0	1	1	0	1	1	1	1	0	1	1	1
12	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	0	1	0	0	1	0	0	1	1	0	1	1	0	0
13	1	0	1	1	0	0	0	0	0	0	0	1	1	1	0	1	1	1	1	0	1	0	0	1	0	0	1	0	1
14	1	1	0	0	0	0	0	0	0	0	0	1	1	0	1	1	0	0	1	0	0	0	0	0	1	0	1	1	0
15	0	1	0	1	0	1	0	1	0	1	0	0	1	0	0	1	0	1	1	1	1	1	1	0	1	1	0	0	0
16	0	1	0	0	0	1	0	0	0	1	0	1	1	1	1	0	1	1	0	0	1	1	0	1	1	0	1	0	1
17	0	0	0	0	0	1	0	1	0	0	0	1	1	0	0	0	1	1	1	1	0	0	0	1	1	1	0	1	1
18	1	1	1	1	1	0	1	1	1	1	1	0	0	0	0	0	1	1	0	0	0	0	0	1	1	0	1	1	0
19	0	1	1	0	0	0	1	1	0	0	1	0	1	0	1	0	0	1	0	1	0	1	0	0	1	0	0	1	0
20	1	1	0	1	0	1	1	0	1	0	0	0	1	0	0	0	0	1	0	0	0	1	0	1	1	1	1	0	1
21	1	0	1	1	1	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	0	0	1	1	0	0	0	1
22	1	1	0	0	1	0	1	0	0	0	1	0	1	0	1	0	1	1	1	0	1	0	1	1	0	1	0	1	1
23	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1	0	0	1	1	1	0	1	1	0	1	1	1	0	0
24	0	0	0	0	1	0	1	0	1	0	0	0	0	0	1	0	1	1	0	0	0	1	1	1	0	0	1	0	1
25	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	1	0	1	0	1	1	0	0	0	0	1	0	0
26	0	1	0	0	1	1	1	0	0	1	1	0	0	1	0	0	1	0	0	0	1	1	1	1	1	1	0	1	0
27	1	1	1	1	0	0	1	0	1	1	0	1	0	0	0	0	1	0	1	0	0	1	1	0	0	1	0	0	0
28	1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	1	1	1	0	1	0	1
29	1	0	1	0	1	0	0	0	0	1	0	1	0	1	0	1	1	1	1	1	1	1	0	0	1	1	1	0	1

Fase 3 — gedemaskeerde matrix B Matrix B (459 enen)

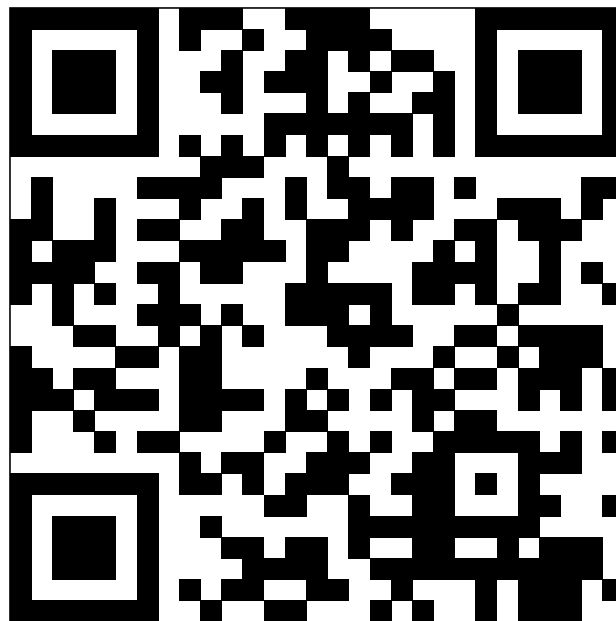
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
1	0	0	1	1	1	1	1	1	0	0	0	1	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	1
2	1	1	0	0	0	1	1	0	1	0	0	0	0	0	1	1	1	1	1	1	0	0	1	1	0	0	0	1	1
3	0	0	1	1	0	0	0	0	0	1	1	1	1	1	0	1	1	0	0	1	1	1	1	0	1	1	1	1	1
4	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	1	0	1	0	0	1
5	0	1	0	0	0	0	0	0	1	1	0	1	0	0	0	0	1	0	0	0	1	0	0	0	1	1	0	1	0
6	1	0	1	1	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0	1	1	0	1	1	1	0	0
7	0	1	0	1	1	1	0	1	0	1	0	0	0	0	1	0	0	1	0	0	1	1	1	1	0	1	1	0	0
8	0	1	1	1	1	1	0	0	0	0	1	0	0	1	1	0	0	1	1	0	1	1	0	0	0	1	0	0	0
9	0	0	1	0	1	0	0	1	0	1	1	0	0	1	1	1	1	1	0	1	1	1	0	1	1	1	0	0	1
10	1	0	1	1	1	0	0	0	1	1	0	0	1	0	1	0	0	0	1	0	1	1	1	1	1	0	0	1	1
11	1	1	0	1	1	1	1	1	0	0	1	0	1	0	1	1	1	0	1	0	1	0	1	1	1	0	0	1	0
12	0	0	0	1	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	0	1	1	1	1	1	0	1	1	0
13	0	0	0	1	1	1	1	1	0	0	0	0	0	1	1	1	1	0	1	1	1	0	1	0	1	1	1	0	1
14	1	0	1	1	1	0	0	1	0	1	0	0	1	0	1	1	1	0	0	1	0	1	1	0	0	0	1	0	1
15	0	1	0	1	0	1	1	0	1	0	1	0	0	1	0	0	1	0	1	1	1	0	1	0	1	1	1	0	1
16	1	0	1	1	0	0	1	0	1	0	1	0	1	1	1	1	1	1	0	1	1	1	0	0	1	0	1	0	1
17	0	1	0	0	1	1	1	0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	1	0	1	1	1	1	1
18	0	1	1	1	1	1	1	0	0	0	1	0	0	0	1	1	1	0	1	1	1	0	1	0	1	1	1	1	1
19	1	0	1	0	1	0	1	0	0	1	1	0	1	1	0	0	1	1	0	1	1	1	1	1	1	0	1	0	1
20	1	1	1	0	1	1	0	1	1	1	1	0	1	1	0	1	0	1	1	0	1	1	0	0	1	1	0	0	1
21	0	0	0	1	1	0	1	1	1	1	0	1	0	1	1	0	0	0	1	1	1	0	1	0	0	0	1	1	0
22	0	0	1	0	0	1	1	0	0	0	0	0	1	1	1	0	1	1	1	0	0	1	0	1	1	0	0	1	1
23	1	0	0	0	1	1	1	1	1	1	0	1	0	0	0	0	0	1	1	1	0	0	1	0	1	1	0	1	0
24	0	0	1	0	1	1	1	1	1	0	1	1	1	0	0	1	0	1	0	0	0	1	0	1	1	1	0	1	0
25	1	1	0	0	0	0	1	1	0	0	1	0	0	1	0	0	1	1	0	0	1	1	1	0	0	1	1	1	0
26	0	1	0	0	1	0	1	1	1	0	1	0	0	0	1	1	1	1	1	0	0	0	0	1	0	1	1	1	1
27	1	0	1	0	1	0	1	0	1	1	1	1	1	0	0	0	0	0	1	0	1	1	1	0	1	0	1	0	0
28	1	1	0	0	0	1	1	1	0	1	1	0	0	0	1	1	0	0	0	0	0	1	0	1	1	0	1	1	0
29	0	0	0	1	1	1	1	1	1	0	1	1	0	0	1	1	1	1	0	1	0	1	0	1	0	1	1	0	1

Fase 4 — oplossing X

De volledige oplossing X , numeriek:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
1	1	1	1	1	1	1	1	0	1	1	1	0	0	1	1	1	1	0	0	0	0	0	1	1	1	1	1	1	1
2	1	0	0	0	0	0	1	0	1	0	0	1	0	0	1	1	1	1	0	0	0	0	1	0	0	0	0	0	1
3	1	0	1	1	1	0	1	0	0	0	1	0	0	1	1	0	1	0	1	1	0	0	1	0	1	1	1	0	1
4	1	0	1	1	1	0	1	0	0	1	1	0	1	1	1	0	0	1	0	0	1	0	1	0	1	1	1	0	1
5	1	0	1	1	1	0	1	0	0	0	1	0	0	0	1	0	1	0	0	1	1	0	1	0	1	1	1	0	1
6	1	0	0	0	0	0	1	0	0	0	0	0	1	1	1	1	1	1	1	0	1	0	1	0	0	0	0	0	1
7	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1	1
8	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	1	0	0	0	0	1	1	1	1	0	0	0	1	0	1	1	0	1	1	0	1	0	0	0	0	0	0	1
10	0	1	0	0	1	0	0	0	1	0	0	1	0	1	0	0	1	1	1	1	0	0	0	1	1	1	0	1	0
11	1	1	0	0	1	0	1	1	0	0	1	1	0	0	1	1	1	0	1	0	0	1	1	0	1	1	1	0	0
12	1	0	1	1	1	0	0	0	1	1	1	1	1	0	1	1	0	0	0	0	1	0	1	0	0	1	0	0	0
13	1	1	1	1	0	0	1	1	0	1	1	0	0	0	1	0	0	0	0	1	1	1	1	0	0	1	0	1	0
14	1	0	1	0	1	0	0	1	0	0	0	0	0	1	1	1	1	1	0	1	1	0	1	0	1	1	0	0	1
15	0	0	1	0	0	0	1	1	1	0	1	0	0	1	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0
16	0	0	0	1	1	0	0	1	0	1	0	1	0	0	0	1	1	1	1	0	0	1	1	0	1	1	1	1	0
17	1	0	0	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	1	1	1	0
18	1	1	0	1	1	0	0	0	1	1	0	0	0	0	1	0	0	0	1	1	0	1	1	0	1	1	1	0	1
19	1	1	1	0	1	1	1	0	1	1	1	1	0	1	0	1	0	1	0	1	1	0	0	0	1	0	1	0	1
20	1	0	0	1	0	0	0	0	1	1	0	0	0	1	1	0	0	0	0	1	0	1	1	0	0	0	0	0	0
21	1	0	0	0	1	0	1	1	0	0	0	0	1	0	0	1	0	1	0	0	1	1	1	1	1	0	1	0	1
22	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1	1	1	0	0	1	1	0	0	0	1	1	1	0	0
23	1	1	1	1	1	1	1	0	1	1	0	1	0	1	1	1	0	0	1	1	1	0	1	0	1	0	0	0	0
24	1	0	0	0	0	0	1	0	0	0	0	0	1	1	0	0	1	0	0	1	1	0	0	0	1	1	0	0	1
25	1	0	1	1	1	0	1	0	0	1	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	0	0	0	0
26	1	0	1	1	1	0	1	0	0	0	1	1	0	1	1	0	1	0	0	1	1	0	0	1	0	1	1	1	0
27	1	0	1	1	1	0	1	0	0	0	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	0
28	1	0	0	0	0	0	1	0	1	0	1	1	0	0	1	1	0	0	1	0	0	0	1	0	1	1	1	0	1
29	1	1	1	1	1	1	1	0	0	1	0	0	1	1	0	0	0	1	0	1	0	0	1	0	1	1	1	0	0

Dezelfde oplossing, visueel weergegeven (1 = zwart, 0 = wit) — dit *is* letterlijk de QR-code, klaar om over te tekenen met een witte rand van minstens 4 cellen eromheen:



Gecontroleerd en bevestigd scanbaar. De code leidt naar de opgegeven link.